

라즈베리파이와 SenseHAT으로 배우는 피지컬 컴퓨팅

초·중·고 교사 연수 · 3차시(50분 × 3 = 150분 연강) 강사: **오정훈** (부산동고등학교)

1. 연수 목표

- 1. 라즈베리파이에 운영체제를 직접 설치하고, 한글 환경을 설정할 수 있다.
- 2. SenseHAT의 LED·조이스틱·센서를 Python과 Scratch로 제어할 수 있다.
- 3. 변수·반복·조건 같은 파이썬 기초 문법을 한 번에 하나씩, 짧은 코드와 "눈에 보이는 결과"로 익힌다.
- 4. 배운 내용을 자기 학교급 수업에 적용할 아이디어를 얻는다.

2. 시간표

차시	시간	주제	주요 활동
1차시	0~50분	시작하기	SD카드 굽기, 라즈베리파이·SenseHAT 소개, 한글 환경 설정, 첫 코드
2차시	50~100분	SenseHAT으로 배우는 파이썬 기초	12개 작은 단계(s01~s12), 단계마다 개념 1개·코드 1~6줄, Python ↔ Scratch 비교
3차시	100~150분	미니 프로젝트와 수업 적용	쉬운 프로젝트 택1(p01~p03), 학교급별 적용, 정리

150분 연강이라 휴식 시간이 따로 없습니다. SD카드를 굽는 시간(1차시)과 프로젝트 시간(3차시)에 자유롭게 쉬도록 안내합니다.

3. 준비물 (1~2인 1세트)

품목	비고
Raspberry Pi 4B (8GB)	
Argon One V2 케이스	상단 GPIO 커버를 열고 SenseHAT을 장착

품목	비고
Sense HAT (V2)	V2에는 컬러 센서가 있음. V1이면 컬러 센서 프로젝트(p02) 제외
microSD 카드 32GB 이상 + 카드 리더기	참가자 노트북/PC에서 굽기
USB-C 전원 어댑터 (5V 3A)	전원이 약하면 번개 모양 경고가 뜸
모니터 + 일반(표준) HDMI 케이블	Argon One V2는 HDMI가 뒤쪽 폴사이즈 포트로 나와서 micro-HDMI 케이블이 필요 없음
키보드·마우스	한/영 키가 있는 한글 키보드 권장
참가자 PC (Windows/macOS)	Raspberry Pi Imager 미리 설치

4. 강사 사전 체크리스트

- ☐ 연수장 Wi-Fi SWAI_5G / swai2024 가 켜져 있는지 확인하고 칠판에도 적기 (5GHz라 공유기와 너무 멀면 약할 수 있음)
- ☐ **Raspberry Pi OS Full (64-bit)** 이미지 파일(.img.xz)을 미리 받아 USB 메모리 여러 개에 복사
 - 파일: 2026-09-15-raspbian-trixie-arm64-full.img.xz (약 2GB)
 - 직접 받기: https://downloads.raspberrypi.com/raspbian_full_arm64/images/raspbian_full_arm64-2026-09-15/2026-09-15-raspbian-trixie-arm64-full.img.xz - 참가자 PC가 각자 인터넷으로 받으면 Wi-Fi가 느려져 시간이 크게 늘어납니다
- ☐ 예비 SD카드를 참가자 수의 20% 이상 미리 구워 두기 (굽기 실패·시간 부족 대비)
- ☐ 예비 SD카드에는 한글 폰트·입력기 설치, 한국어 화면 설정, 입력기에 한글 추가까지 해 두기
- ☐ 참가자 PC에 Raspberry Pi Imager 설치 안내 메일 발송 (<https://www.raspberrypi.com/software/>)
- ☐ 실습 예제 폴더(code/)를 USB 메모리 또는 공유 링크로 준비
- ☐ SenseHAT 장착 상태 점검: 부팅 시 LED에 무지개가 잠깐 켜졌다 꺼지면 정상
- ☐ 강사 기기로 전체 예제 한 번씩 실행해 보기

5. 자주 생기는 문제와 해결

증상	원인	해결
부팅 시 무지개 LED가 안 켜짐	SenseHAT 핀이 한 칸 밀림	전원을 끄고 40핀 전체가 맞게 꽂혔는지 확인

증상	원인	해결
OSError / SenseHAT을 못 찾음	HAT 인식 실패	재부팅, 핀 접촉 확인
온도가 실제보다 10°C 이상 높 음	CPU 열이 센서에 전달됨	정상 현상. 3차시 "센서 보정"에서 다룸
메뉴·화면이 영어로 나옴	Imager의 Seoul 설정은 시간대·키보드만 바꿈	Preferences → Control Centre → Localisation → Set Locale에서 ko (Korean) 선택 후 재부팅
한글 입력이 안 됨	입력기에 한글을 추가하지 않음	키보드 아이콘 오른쪽 클릭 → 설정 → 한글 추가 → 적용. 한/영 키가 안 되면 Ctrl + Space
LED에 한글이 안 나옴	show_message 는 영문·숫자만 지원	영문으로 쓰거나 픽셀로 직접 그림
화면이 안 나옴	HDMI 포트 문제	케이스 뒤쪽의 다른 HDMI 포트에 꽂고 재부팅

6. 자료 구성

```
sensehat-workshop/
├─ docs/           교사용 교재 (이 문서들)
├─ code/python/    s01~s12 기초 단계, p01~p03 프로젝트
│   └─ extra/      추가 예제 e01~e12 (교재 부록)
├─ code/scratch/   Scratch 3 예제 (.sb3)
└─ slides/         발표 슬라이드
```

1차시 · 시작하기 (50분)

시간	활동
0~5분	인사, 연수 안내
5~15분	SD카드 굽기 시작 (Raspberry Pi Imager)
15~30분	굽는 동안: 라즈베리파이·SenseHAT 소개
30~45분	첫 부팅, 한글 폰트·입력기 설치
45~50분	첫 코드: LED에 글자 흘리기

1. SD카드 굽기 (참가자 PC에서)

굽는 데 10~15분쯤 걸립니다. **먼저 시작해 두고** 기다리는 동안 설명을 듣습니다.

어떤 OS를 설치하나요?

Raspberry Pi OS Full (64-bit) — Scratch 3, Thonny, SenseHAT 라이브러리가 모두 들어 있어서 따로 설치할 필요가 없습니다.

버전	Scratch 3	이번 연수
Raspberry Pi OS Full (64-bit)	○ 포함	이것을 사용
Raspberry Pi OS (64-bit)	× 따로 설치	사용하지 않음
Raspberry Pi OS Lite	× 데스크톱 없음	사용하지 않음

◆ Full 버전은 파일이 커서(약 2GB) 모두가 동시에 인터넷으로 받으면 오래 걸립니다. 강사가 나눠 준 **USB 메모리 속 이미지 파일** 2026-09-15-raspios-trixie-arm64-full.img.xz 을 사용합니다.

굽는 순서

왼쪽의 **설정 단계** 목록에서 지금 어느 단계인지 보입니다. 각 화면에서 입력하고 **다음**을 누릅니다.

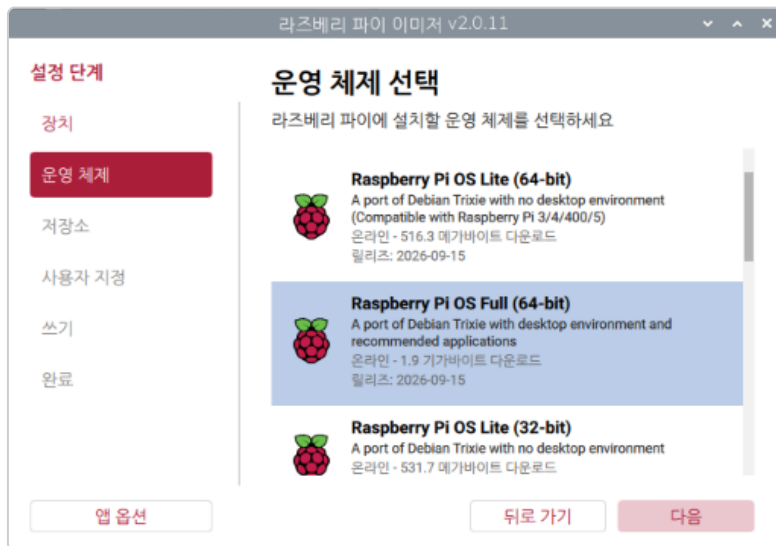
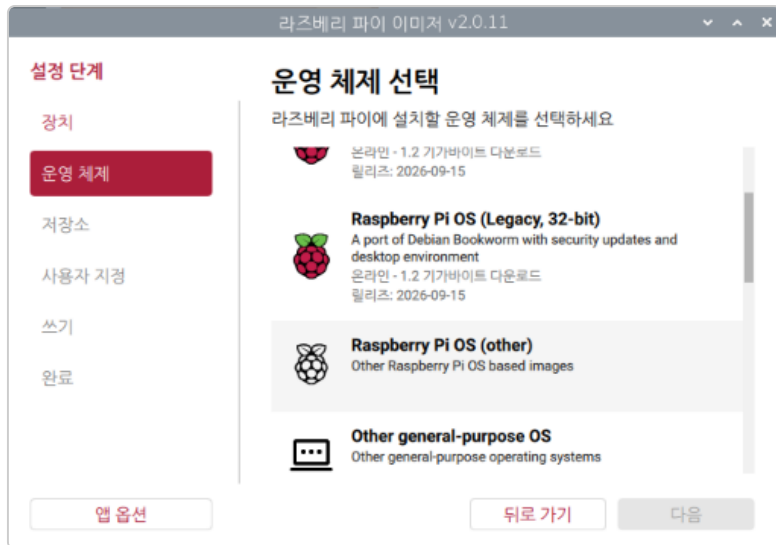
① **준비**: 강사가 나눠 준 USB 메모리에서 이미지 파일을 PC 바탕화면에 복사하고, Raspberry Pi Imager를 실행합니다.

② **장치** → Raspberry Pi 4 선택

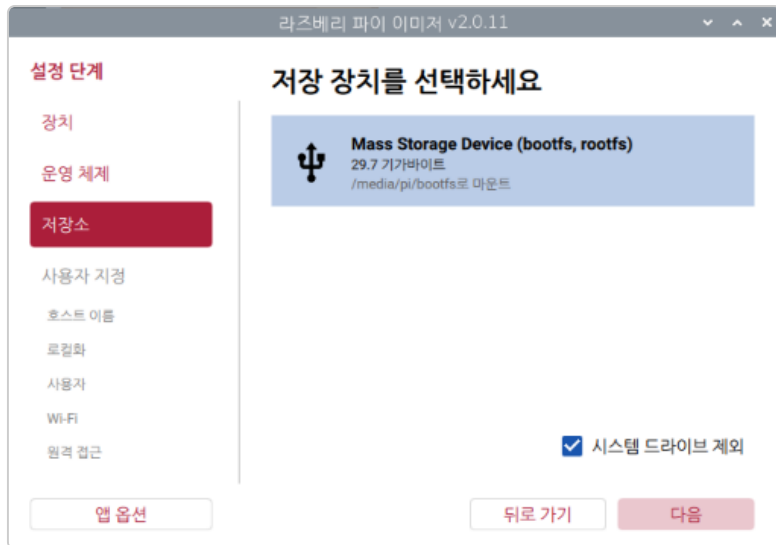


③ **운영 체제** → 목록 맨 아래 **사용자 정의 이미지 사용(Use custom)** → 복사한 이미지 파일 선택

- 인터넷으로 받을 때는 Raspberry Pi OS (other)에 들어가서 Raspberry Pi OS Full (64-bit)을 고릅니다. (맨 위 기본 항목이 아님!)



④ 저장소 → 내 SD카드 선택 (USB 메모리 등 다른 저장장치를 고르지 않도록 주의!)



⑤ 사용자 지정 (6개 화면)

화면	입력
호스트 이름	raspberrypi (기본값 그대로)
지역화	수도 도시에 Seoul 검색 → Seoul (South Korea) 선택 → 시간대 Asia/Seoul , 키보드 kr 이 자동으로 채워짐
사용자 이름	pi / 비밀번호 11111111 (1을 8개, 두 번 입력)
Wi-Fi	SSID SWAI_5G / 비밀번호 swai2024 (두 번 입력, 대소문자 정확히)
SSH 인증	켜지 않음 (그대로 다음)
라즈베리 파이 커넥트	켜지 않음 (그대로 다음)

라즈베리 파이 이미지 v2.0.11

장치

운영 체제

저장소

사용자 지정

호스트 이름

로컬화

사용자

Wi-Fi

원격 접근

라즈베리 파이 커넥트

사용자 지정: 호스트 이름 선택

호스트 이름을 입력하세요

raspberrypi

호스트 이름은 네트워크에서 라즈베리 파이를 식별하는 고유한 이름입니다.
영문자, 숫자 및 하이픈만 포함해야 합니다

앱 옵션

사용자 지정을 건너뛰기

뒤로 가기

다음

라즈베리 파이 이미지 v2.0.11

운영 체제

저장소

사용자 지정

호스트 이름

로컬화

사용자

Wi-Fi

원격 접근

라즈베리 파이 커넥트

쓰기

사용자 지정: 지역화

추천 시간대와 키보드 레이아웃을 위해 위치를 선택하세요

수도 도시: Seoul (South Korea) ⓘ

시간대: Asia/Seoul

키보드 레이아웃: kr

앱 옵션

사용자 지정을 건너뛰기

뒤로 가기

다음

라즈베리 파이 이미지 v2.0.11

사용자 지정: 사용자 이름 선택

라즈베리 파이 계정을 만듭니다

저장소

- 사용자 지정
- 호스트 이름
- 로컬화
- 사용자
- Wi-Fi
- 원격 접근
- 라즈베리 파이 커넥트
- 쓰기
- 완료

사용자 이름:

비밀번호:

비밀번호 확인:

비밀번호 재입력

사용자 이름은 소문자이며 영문자, 숫자, 밑줄 및 하이픈만 포함해야 합니다

앱 옵션 사용자 지정을 건너뛰기 뒤로 가기 다음

라즈베리 파이 이미지 v2.0.11

사용자 지정: Wi-Fi 선택

보안 네트워크 공개 네트워크

SSID:

비밀번호:

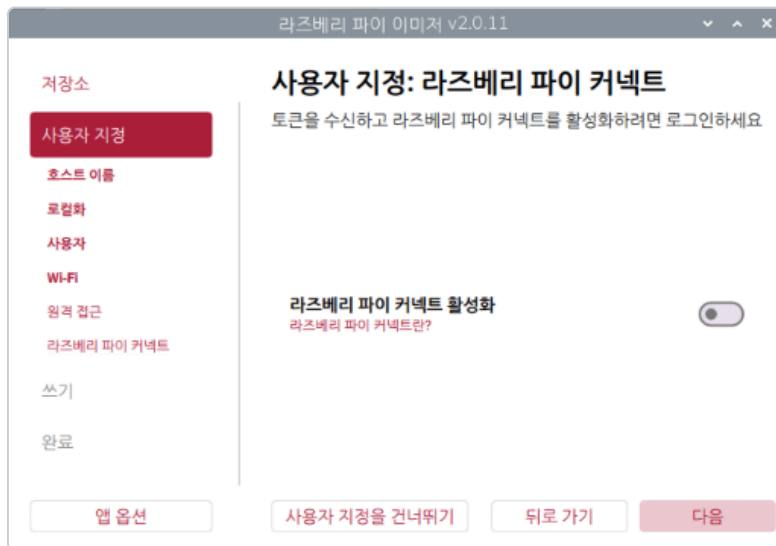
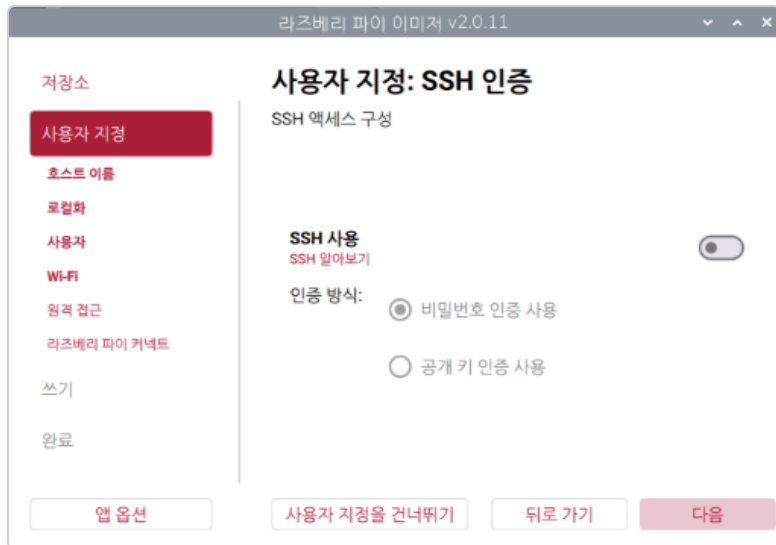
비밀번호 확인:

비밀번호 재입력

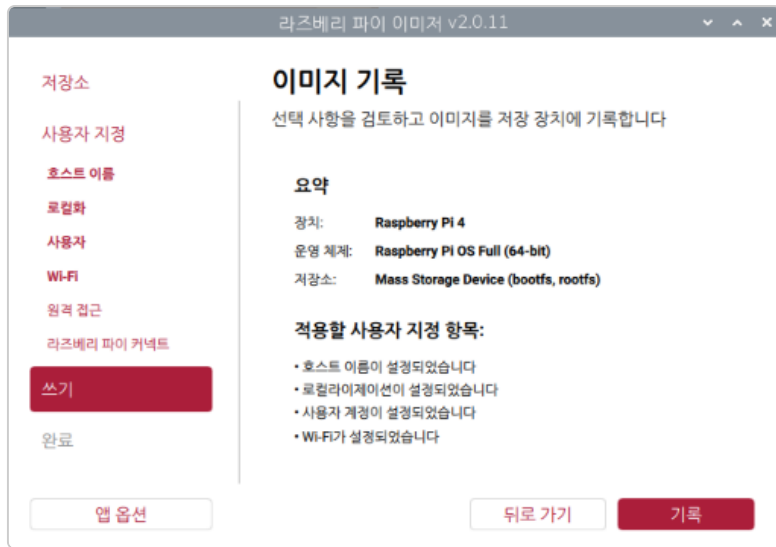
☐ 숨겨진 SSID

앱 옵션 사용자 지정을 건너뛰기 뒤로 가기 다음

△ Wi-Fi 화면에는 **지금 PC가 연결된 Wi-Fi 이름**이 미리 채워져 있을 수 있습니다. 지우고 **SWAI_5G**를 입력하세요. ※ 비밀번호가 8자보다 짧으면 "비밀번호가 너무 짧습니다(최소 8자)"가 뜹니다.



⑥ 쓰기 → 요약을 확인하고 기록 클릭 → 완료되면 SD카드를 빼서 라즈베리파이에 꽂습니다.



⚠ 기록하면 SD카드의 기존 내용이 모두 지워집니다. ♦ 11111111 은 연수용 비밀번호입니다. 학교 수업에서 쓸 때는 바꾸세요 (터미널에서 `passwd`).

참고 영상: OS 설치 (Imager 2.0 기준, 2025년 11월)

- [How to Install Raspberry Pi OS Using Raspberry Pi Imager 2.0](#) — Gadgets Pod, 초보자용 단계별 설명 (영어)
- [Raspberry Pi Imager 2.0: Set Up Your Raspberry Pi in Minutes](#) — Gary Explains (영어)

※ Imager는 2025년 10월 **2.0**으로 크게 바뀌었습니다. 그 전 영상(1.x)은 화면이 달라서 헷갈릴 수 있으니 날짜를 확인하세요.

2. 라즈베리파이 소개 (굽는 동안)

피지컬 컴퓨팅이란?

- 컴퓨터가 센서로 세상을 느끼고(입력), LED·소리·모터로 반응하는(출력) 것
- 화면 속 코딩 → 손에 잡히는 결과로 이어져 학생 흥미가 높습니다.

[센서: 온도·기울기·버튼] → [라즈베리파이: 프로그램] → [LED·화면·소리]

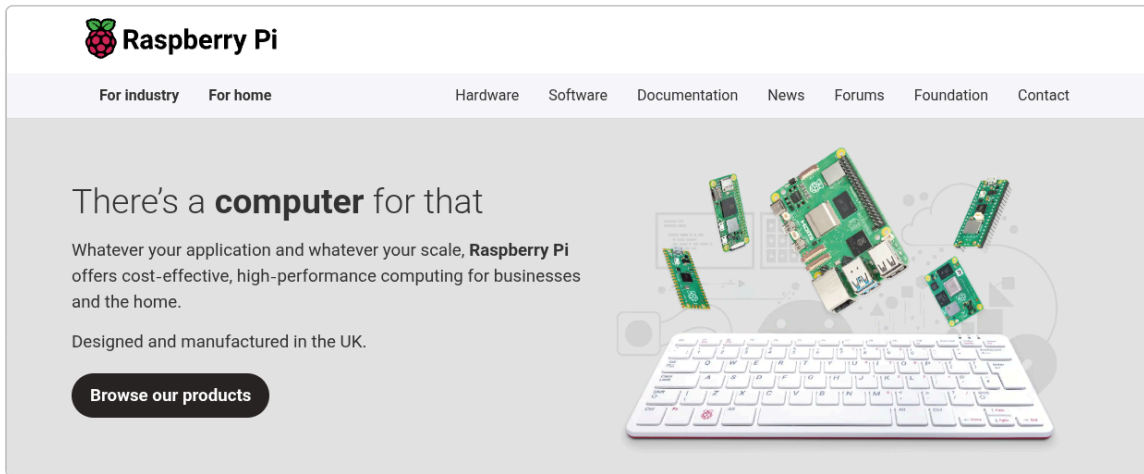
입력

처리

출력

공식 홈페이지

- <https://www.raspberrypi.com> — 라즈베리파이 회사: 제품, 운영체제 다운로드, 설명서
- <https://www.raspberrypi.org> — 라즈베리파이 재단: 교사용 수업 자료, 교사 연수, 코드 클럽



- 영국에서는 초등학생부터 라즈베리파이로 컴퓨팅을 배웁니다. 재단은 수업 자료와 교사 연수를 무료로 제공합니다.

Teaching resources

Computing is a vital aspect of today's world. We want to empower teachers to give young people the skills they need to shape this world.

We provide high-quality CPD training courses, classroom resources, and online events for teachers. All for free.

Subscribe to our newsletter

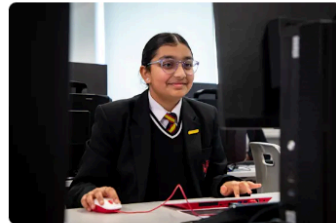


For the classroom

These resources are designed for teachers to use in the classroom. They contain everything you need, like lesson plans, worksheets, homework, and exam preparation.



Computing aimed at ages 5 to 16



Computer science aimed at ages 14 to 19



AI and machine learning

Developed in collaboration with Google DeepMind. Experience AI offers resources and

라즈베리파이의 역사

- 영국 케임브리지 대학의 에벤 업튼(Eben Upton) 등이 "학생들이 싸고 쉽게 프로그래밍을 배울 컴퓨터"를 만들기로 하고, 2009년 라즈베리파이 재단(자선단체)을 세움

연도	사건
2012	Raspberry Pi 1 출시 (35달러)
2015	Raspberry Pi 2, 5달러짜리 Pi Zero / 우주정거장(ISS)에 Astro Pi (SenseHAT 장착)가 올라감
2016	Raspberry Pi 3 (Wi-Fi·블루투스 내장)
2019	Raspberry Pi 4 (오늘 쓰는 보드)
2021	Raspberry Pi Pico (마이크로컨트롤러)
2023	Raspberry Pi 5

- 지금까지 전 세계에서 **수천만 대**가 팔린, 교육용 컴퓨터의 대표

Raspberry Pi 4B 제원

- 신용카드만 한 **완전한 컴퓨터** (리눅스 운영체제, 인터넷, 문서 작업 가능)



항목	내용
CPU	4코어 ARM Cortex-A72, 64비트, 1.8GHz (2019년 초기 보드는 1.5GHz)
메모리	8GB (1·2·4GB 모델도 있음)
무선	Wi-Fi (2.4GHz·5GHz), 블루투스 5.0
유선	기가비트 이더넷
USB	USB 3.0 × 2, USB 2.0 × 2
화면	micro-HDMI × 2 (4K 지원). Argon One V2 케이스에서는 일반 HDMI 로 바뀜
저장	microSD 카드 (운영체제가 여기에 설치됨)
전원	USB-C, 5V 3A
GPIO	40핀 : 센서나 확장보드(HAT)를 연결하는 핀 → 오늘 SenseHAT을 꽂는 곳

가격: 35달러 컴퓨터가 비싸진 이유

- 출시 때 Pi 4 가격 (세금·배송비 제외, 미국 달러)

메모리	출시 가격	2026년 4월 이후
1GB	35달러 (2019년 6월)	35달러 (그대로)
4GB	55달러 (2019년 6월)	100달러
8GB	75달러 (2020년 5월)	165달러

- 2025년 12월 · 2026년 2월 · 2026년 4월, 세 번에 걸쳐 값이 올랐습니다.
- 이유: **AI 데이터센터**가 고성능 메모리를 대량으로 쓰면서 메모리 공장이 그쪽 생산에 몰렸고, 라즈베리파이가 쓰는 LPDDR4 메모리 값이 1년 사이 **약 7배** 올랐습니다. (라즈베리파이 공식 발표, 2026년 4월)
- 그래서 메모리가 큰 모델일수록 많이 올랐고, 1GB 모델은 그대로입니다. 재단은 메모리 값이 내려가면 가격을 되돌리겠다고 밝혔습니다.
- 수업 이야깃거리: "AI가 발전하면 왜 작은 컴퓨터 값이 오를까?" → 반도체 수요와 공급

소개 영상

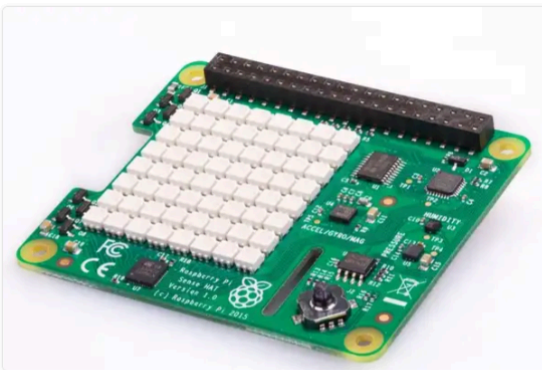
- [What is a Raspberry Pi?](#) — 라즈베리파이 공식 채널 (영어)
- [What is a Raspberry Pi 4](#) — 라즈베리파이 공식 채널 (영어)
- [라즈베리 파이 소개 및 설치 방법](#) — 다두이노 채널 (한국어)

Argon One V2 케이스

- 알루미늄 케이스 + 냉각 팬
- 포트가 뒤쪽으로 모여 있음 (HDMI 2개는 일반 HDMI 크기)
- 위쪽 자석 덮개를 열면 GPIO 핀이 나옴 → 여기에 SenseHAT 장착
- 전원 버튼: 한 번 누르면 켜짐, 길게 누르면 꺼짐

3. SenseHAT 소개

원래 국제우주정거장(ISS) 교육 프로젝트 **Astro Pi**용으로 만든 확장보드입니다.



Sense HAT

The Sense HAT is an add-on board that gives your Raspberry Pi an array of sensing capabilities. The on-board sensors allow you to monitor pressure, humidity, temperature, colour, orientation, and movement. The bright 8x8 RGB LED matrix allows you to visualise data from the sensors, and the five-button joystick lets users interact with your projects.



Buy now

Hardware designed for space

The Sense HAT was originally developed for use on the International Space Station, as part of the educational [Astro Pi](#) programme run by the Raspberry Pi Foundation in partnership with the European Space Agency. It is well suited to many projects that require position, motion, orientation, or environmental sensing. The Sense HAT is powered by the Raspberry Pi computer to which it is connected.

An officially supported [Python library](#) provides access to all of the on-board sensors, the LED matrix, and the joystick. The Sense HAT is compatible with any Raspberry Pi computer with a 40-pin GPIO header.

부품	하는 일
8×8 RGB LED	64개 점을 원하는 색으로 켜 → 글자·그림 표시
조이스틱	위·아래·왼쪽·오른쪽·누르기 (5방향 버튼)
온도·습도 센서	주변 온도(°C), 습도(%)
기압 센서	기압(hPa)
움직임 센서	기울기, 흔들림, 방향(나침반)
컬러 센서 (V2만)	앞에 댄 물체의 색(빨강·초록·파랑 양)

LED 좌표

```
x → 0 1 2 3 4 5 6 7
y 0   ● . . . . . . . ← (0, 0)은 왼쪽 위
  ↓ 1   . . . . . . .
    ...
  7   . . . . . . . ● ← (7, 7)은 오른쪽 아래
```

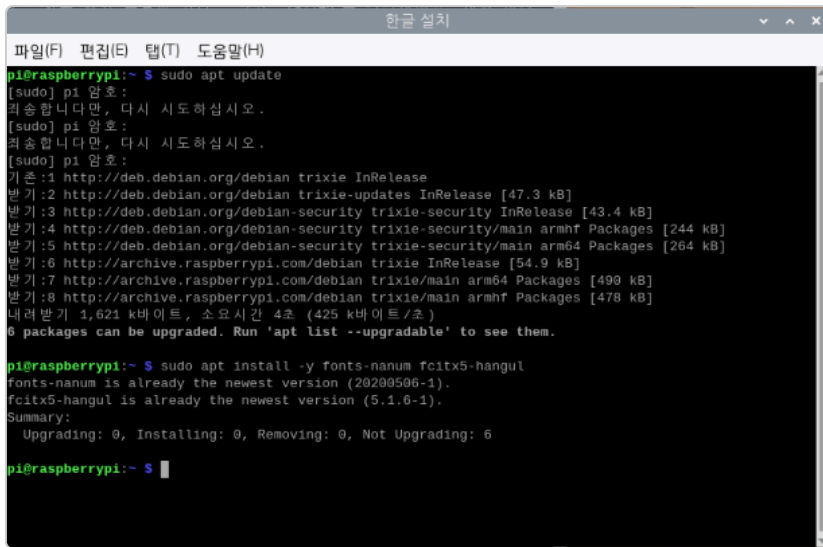
4. 첫 부팅과 한글 환경 설정

1. SD카드를 꽂고 전원 연결 → SenseHAT에 **무지개 LED가 잠깐 켜지면** 정상
2. 바탕화면이 나오면 Wi-Fi 연결 확인 (오른쪽 위 아이콘)
3. 터미널을 열고(검은 화면 아이콘) 아래 명령을 **한 줄씩** 입력합니다.

```
sudo apt update
sudo apt install -y fonts-nanum fcitx5-hangul
```

명령	뜻
<code>sudo apt update</code>	설치할 프로그램 목록을 최신으로 받기 (관리자 권한)
<code>sudo apt install -y fonts-nanum fcitx5-hangul</code>	나눔 글꼴 + 한글 입력기 설치 (-y: 자동으로 "예")

※ `fcitx5-hangul` 만 설치하면 입력기 본체(`fcitx5`)와 설정 도구가 함께 설치되고, 재부팅하면 입력기가 자동으로 켜집니다. 단, **한글은 아래 7번처럼 직접 추가**해야 합니다. 비밀번호를 물으면 `11111111` 입력 (입력해도 화면에 안 보이는 것이 정상)



```
한글 설치
파일(F) 편집(E) 탭(T) 도움말(H)
pi@raspberrypi:~$ sudo apt update
[sudo] pi 암호:
죄송합니다만, 다시 시도하십시오.
[sudo] pi 암호:
죄송합니다만, 다시 시도하십시오.
[sudo] pi 암호:
기존:1 http://deb.debian.org/debian trixie InRelease
받기:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
받기:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
받기:4 http://deb.debian.org/debian-security trixie-security/main armhf Packages [244 kB]
받기:5 http://deb.debian.org/debian-security trixie-security/main arm64 Packages [264 kB]
받기:6 http://archive.raspberrypi.com/debian trixie InRelease [54.9 kB]
받기:7 http://archive.raspberrypi.com/debian trixie/main arm64 Packages [490 kB]
받기:8 http://archive.raspberrypi.com/debian trixie/main armhf Packages [478 kB]
내려받기 1,621 k바이트, 소요시간 4초 (425 k바이트/초)
0 packages can be upgraded. Run 'apt list --upgradable' to see them.

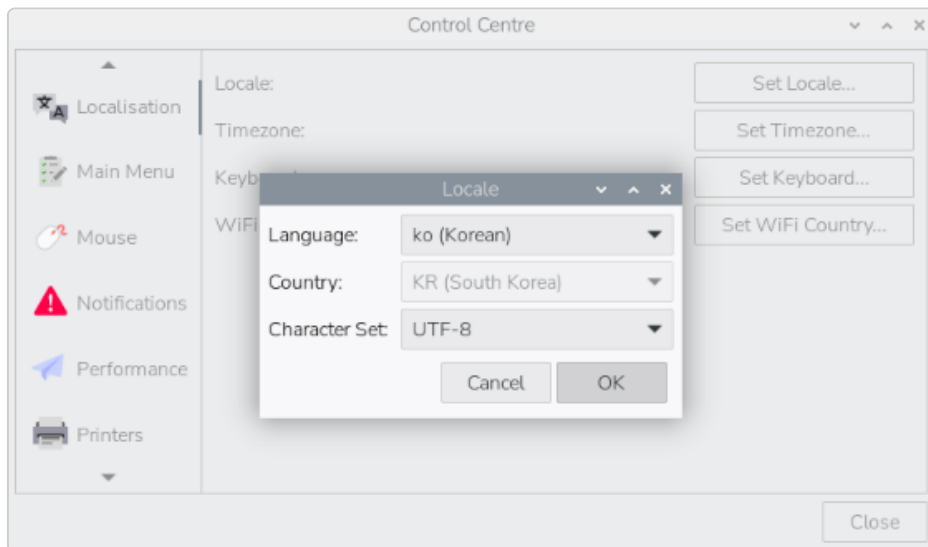
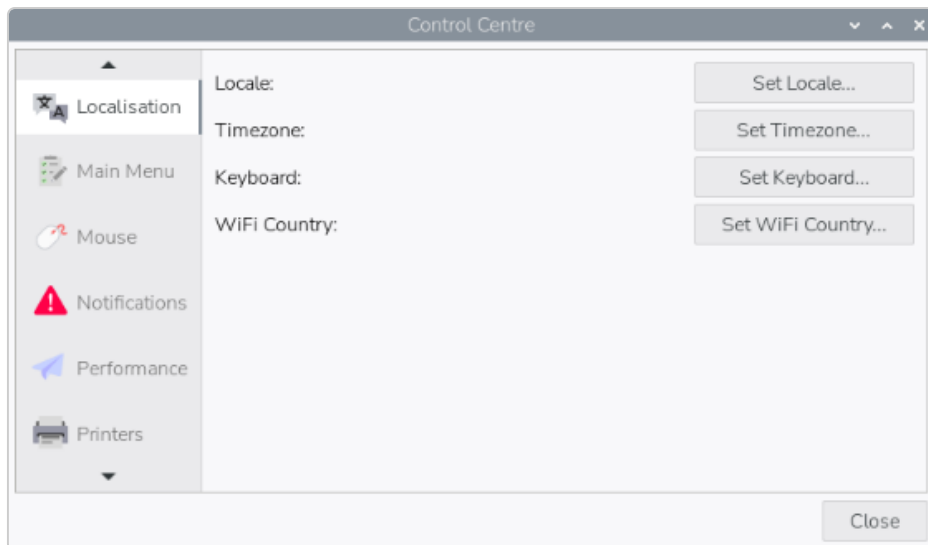
pi@raspberrypi:~$ sudo apt install -y fonts-nanum fcitx5-hangul
fonts-nanum is already the newest version (20200506-1).
fcitx5-hangul is already the newest version (5.1.0-1).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

pi@raspberrypi:~$
```

- [sudo] pi 암호: → 비밀번호를 입력하고 Enter
- 비밀번호가 틀리면 죄송합니다만, 다시 시도하십시오. 가 나옵니다. 다시 입력하면 됩니다.
- 위 그림은 이미 설치된 기기에서 찍어서 already the newest version (이미 최신 버전)이라고 나옵니다. 처음 설치할 때는 내려받기·설치 과정이 여러 줄 지나갑니다.

4. 화면 언어를 한국어로 바꾸기

- Imager에서 Seoul을 골라도 시간대와 키보드만 바뀌고, 메뉴·화면은 영어로 나옵니다.
- 라즈베리 메뉴 → **Preferences** → **Control Centre** → 왼쪽 **Localisation** → **Set Locale...**
- Language ko (Korean) , Country KR (South Korea) , Character Set UTF-8 → **OK** → **Close**



5. 터미널에서 재부팅합니다. 켜지면 메뉴가 한국어로 나옵니다.

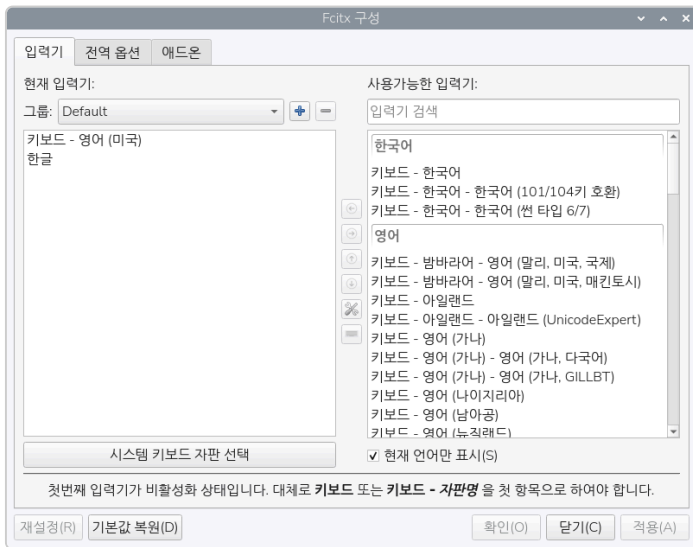
```
sudo reboot
```

※ 글꼴을 먼저 설치하고(3번) 언어를 바꿔야(4번) 한글 메뉴가 깨지지 않습니다.

6. 재부팅 후 오른쪽 위에 **키보드 아이콘**이 보이는지 확인

7. **입력기에 한글 추가** (꼭 해야 함)

- 키보드 아이콘 **오른쪽 클릭** → **설정** → **Fcitx 구성** 창이 열림
- 오른쪽 **사용가능한 입력기의 입력기 검색** 칸에 **한글 입력** → **한글 선택** → 가운데 ◀ 버튼으로 왼쪽 **현재 입력기** 목록에 추가
- **적용** 클릭
- 아래 그림처럼 왼쪽 **현재 입력기**에 **키보드 - 영어 (미국)** 과 **한글** 이 있으면 완료



8. **한/영 키**(또는 **Ctrl + Space**)로 한글 입력을 확인합니다.

◆ 시간이 부족하면 강사가 준비한 **예비 SD카드**(한글 설정 완료본)로 바꿔 끼웁니다.

📺 **한글 환경 설정 영상에 대해:** 2026년 9월 현재, 최신 OS(Trixie)와 fcitx5 기준의 한국어 영상은 찾기 어렵습니다. 검색되는 영상 대부분은 2017~2022년 영상으로, 예전 입력기(ibus, fcitx 4)와 예전 명령을 씁니다. **이 교재의 캡처 순서를 따르는 것이 가장 정확합니다.**

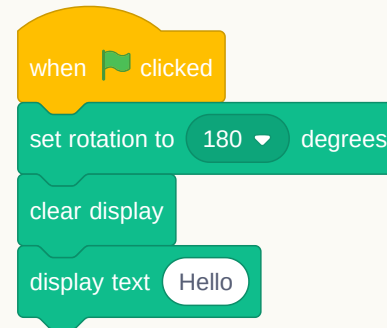
5. 첫 코드 (5분)

메뉴 → 프로그래밍 → **Thonny**를 열고 입력 → **Run(▶)**

Python

```
from sense_hat import SenseHat
sense = SenseHat()
sense.set_rotation(180)
sense.clear()
sense.show_message("Hello")
```

Scratch



- `set_rotation(180)` : Argon 케이스에서는 SenseHAT이 거꾸로 꽂히므로 화면을 180도 돌립니다.
- `clear()` : 화면을 지우고 시작합니다.

LED에 Hello 가 흘러가면 1차시 완료!

※ `show_message` 는 **영문·숫자만** 표시합니다. 한글을 넣으면 깨져 보입니다.

Scratch로는? 메뉴 → 프로그래밍 → **Scratch 3** → 왼쪽 아래 **확장 기능 추가** → Raspberry Pi Sense HAT

2차시 · SenseHAT으로 배우는 파이썬 기초 (50분)

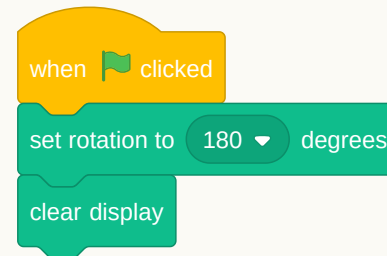
규칙: 한 단계 = 새 개념 1개. 코드를 입력하고 ▶ 실행하고, "바꿔 보기"를 한 가지 해 봅니다. 빨리 끝난 분은 ★ 도전 과제를 해 봅니다.

모든 코드의 첫 네 줄 (항상 똑같음)

Python

```
from sense_hat import SenseHat
sense = SenseHat()
sense.set_rotation(180)
sense.clear()
```

Scratch



줄	뜻
<code>from sense_hat import SenseHat</code>	SenseHAT 도구 상자를 가져오기
<code>sense = SenseHat()</code>	내 SenseHAT을 <code>sense</code> 라는 이름으로 부르기
<code>sense.set_rotation(180)</code>	화면을 180도 돌리기 (Argon 케이스에서는 SenseHAT이 거꾸로 꽂히기 때문)
<code>sense.clear()</code>	화면 지우기 (이전에 켜져 있던 LED 끄기)

아래 예제에서는 이 네 줄을 생략합니다. 파일마다 맨 위에 꼭 넣어 주세요. `sleep` 을 쓰는 단계(s05~)는 `from time import sleep` 도 함께 넣습니다.

Scratch도 초록 깃발 바로 아래에 항상 이 두 블록을 먼저 놓습니다. (아래 예제에서는 생략)

단계	새 개념	결과	시간
s01	명령(함수 호출)	글자 흘리기	3분

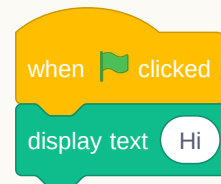
단계	새 개념	결과	시간
s02	변수	글자를 변수에 담기	3분
s03	색(RGB)	빨간 글자	4분
s04	좌표	점 하나 켜기	4분
s05	순서·기다리기	A → B 차례로	4분
s06	반복(for)	한 줄 켜기	5분
s07	리스트	무지개 신호등	4분
s08	센서 읽기	온도 표시	4분
s09	조건(if)	더우면 빨강	4분
s10	무한 반복(while)	계속 재는 온도계	4분
s11	조이스틱	누른 방향 표시	5분
s12	함수 만들기(def)	번쩍!	4분

s01 · 글자 흘리기 — 명령

Python

```
sense.show_message("Hi")
```

Scratch



- `sense.show_message(...)`: "SenseHAT아, 글자를 보여 줘"라는 **명령**
- 글자는 "따옴표" 안에 씁니다(문자열).
- ► 바꿔 보기: "Hi" 대신 내 이름(영문)

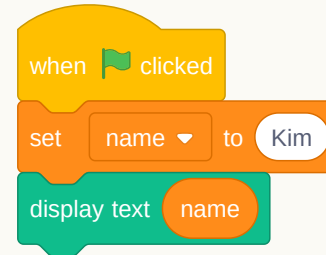
s02 · 변수 — 이름표 붙인 상자

Python

```
name = "Kim"
sense.show_message(name)
```

- `name` 이라는 상자에 "Kim" 을 넣고, 상자를 통째로 넘깁니다.
- ▶ 바꿔 보기: `name` 에 다른 이름 넣기

Scratch



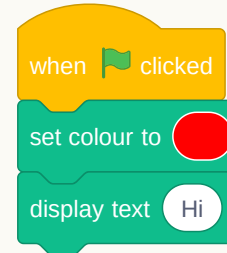
s03 · 색 — 빨강·초록·파랑 섞기

Python

```
sense.show_message("Hi", text_colour=(255, 0, 0))
```

- (빨강, 초록, 파랑) : 각각 0~255
- (255, 0, 0) 빨강, (0, 255, 0) 초록, (255, 255, 0) 노랑
- ▶ 바꿔 보기: 보라색 만들기
- ★ 도전: `scroll_speed=0.05` 를 추가해서 속도 바꾸기

Scratch



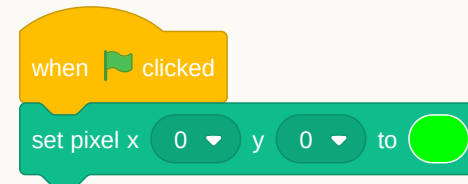
s04 · 좌표 — 점 하나 켜기

Python

```
sense.set_pixel(0, 0, (0, 255, 0))
```

- `set_pixel(x, y, 색)`: 왼쪽 위가 (0, 0), 오른쪽 아래가 (7, 7)
- ▶ 바꿔 보기: 네 모서리에 점 찍기

Scratch



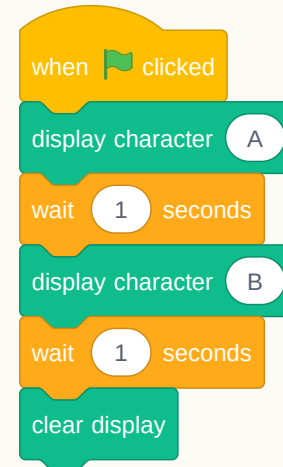
s05 · 순서와 기다리기

Python

```
from time import sleep
sense.show_letter("A")
sleep(1)
sense.show_letter("B")
sleep(1)
sense.clear()
```

- 컴퓨터는 위에서 아래로 한 줄씩 실행합니다.
- `sleep(1)`: 1초 기다리기
- ▶ 바꿔 보기: 3, 2, 1 카운트다운

Scratch

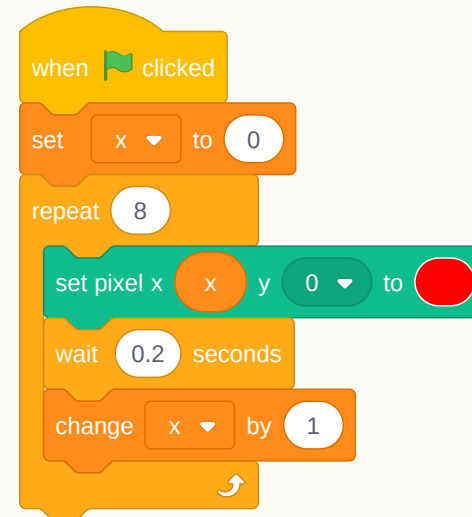


s06 · 반복(for) — 같은 일을 여러 번

Python

```
from time import sleep
for x in range(8):
    sense.set_pixel(x, 0, (255, 0, 0))
    sleep(0.2)
```

Scratch



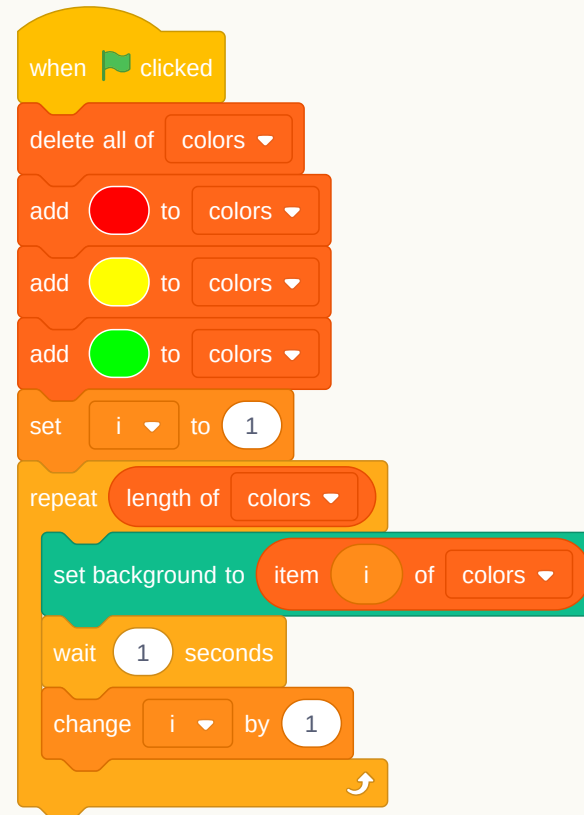
- `range(8)` : 0, 1, 2, ..., 7
- 반복할 줄은 들여쓰기(스페이스 4칸) 합니다.
- ▶ 바꿔 보기: 맨 아래 줄(`y = 7`)을 켜기
- ★ 도전: 세로줄 켜기

s07 · 리스트 — 여러 개를 한 줄로

Python

```
from time import sleep
colors = [(255, 0, 0), (255, 255, 0), (0, 255, 0)]
for c in colors:
    sense.clear(c)
    sleep(1)
```

Scratch



- [...] 대괄호 안에 여러 값을 **순서대로** 담은 것이 리스트
- `for c in colors:` 리스트에서 하나씩 꺼내 `c`에 넣기
- `sense.clear(색)`: 화면 전체를 그 색으로 채우기 (Scratch는 `set background to` 블록)
- ▶ 바꿔 보기: 리스트에 파란색 추가

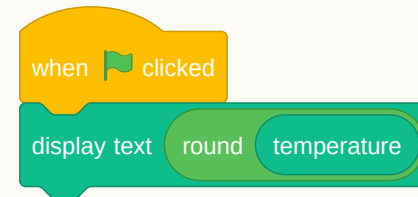
s08 · 센서 읽기 — 온도

Python

```
t = sense.get_temperature()
sense.show_message(str(round(t)))
```

- `get_temperature()`: 온도를 숫자로 알려 줌 (예: 27.8341...)
- `round(t)`: 반올림 → 28
- `str(...)`: 숫자를 글자로 바꿈 (`show_message`는 글자만 받음)
- ▶ 바꿔 보기: `get_humidity()` 로 습도 표시
- ※ 온도가 실제보다 높게 나오나요? → 3차시에서 이유를 알아봅니다.

Scratch

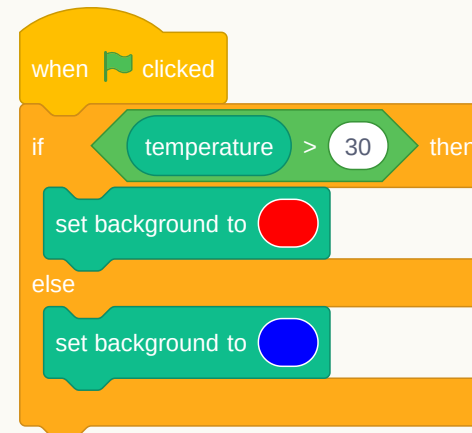


s09 · 조건(if) — 만약 ~라면

Python

```
t = sense.get_temperature()
if t > 30:
    sense.clear(255, 0, 0)
else:
    sense.clear(0, 0, 255)
```

Scratch



- 30도보다 높으면 빨강, 아니면 파랑

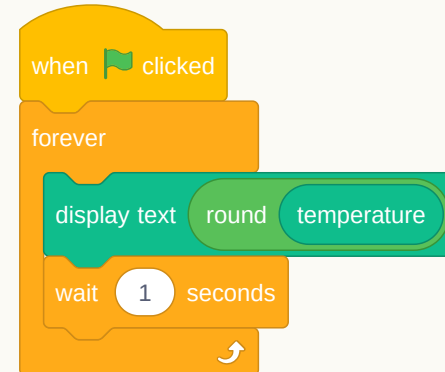
- `if` 줄 끝의 `:` (콜론)과 들여쓰기를 잊지 마세요.
- ▶ 바꿔 보기: 손으로 센서를 감싸서 색이 바뀌는 기준을 찾아보기

s10 · 무한 반복(`while`) — 계속 재기

Python

```
from time import sleep
while True:
    t = sense.get_temperature()
    sense.show_message(str(round(t)))
    sleep(1)
```

Scratch



- `while True:` : 멈출 때까지 계속 반복
- 멈추기: Thonny의 **Stop(■)** 버튼
- ▶ 바꿔 보기: s09의 `if` 를 넣어 온도에 따라 글자색 바꾸기

s11 · 조이스틱 — 누르면 반응

Python

```
while True:
    e = sense.stick.wait_for_event()
    if e.action == "pressed":
        sense.show_message(e.direction)
```

Scratch

when joystick pushed up ▼

display text up

when joystick pushed down ▼

display text down

- `wait_for_event()`: 조이스틱을 움직일 때까지 기다림
- `e.direction`: up, down, left, right, middle
- `==`: "같은가?" (= 하나는 "넣기"이니 구분하세요)
- ► 바꿔 보기: 방향 대신 `show_letter("U")` 같은 글자 한 개
- ⚠ 위로 밀었는데 down 이 나오나요? SenseHAT이 거꾸로 꽂혀 있어서 조이스틱 방향이 반대로 읽힙니다. (`set_rotation(180)` 은 화면만 돌리고 조이스틱은 돌리지 않음. Scratch도 같음) 가운데 누르기(middle)는 그대로입니다.

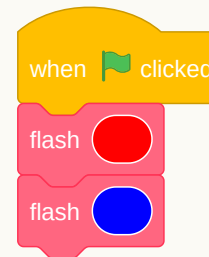
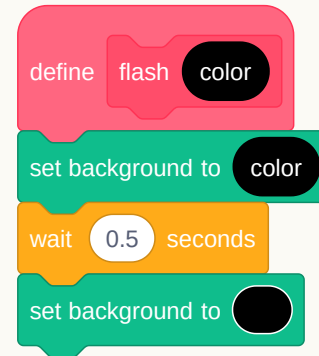
s12 · 함수 만들기(def) — 나만의 명령

Python

```
from time import sleep
def flash(color):
    sense.clear(color)
    sleep(0.5)
    sense.clear()

flash((255, 0, 0))
flash((0, 0, 255))
```

Scratch



- `def` 이름(재료): 여러 줄을 묶어 **새 명령**을 만듭니다.
- 한 번 만들면 `flash(...)` 처럼 몇 번이고 불러 씁니다.
- ▶ 바꿔 보기: 경찰차처럼 빨강·파랑 번갈아 5번 (`for` 사용)

정리: 오늘 배운 문법

문법	한 줄 요약	단계
명령(...)	일 시키기	s01

문법	한 줄 요약	단계
이름 = 값	변수에 담기	s02
(R, G, B)	색 표현	s03
for ... in range()	정해진 횟수만큼 반복	s06
[a, b, c]	리스트	s07
str(), round()	글자로 바꾸기, 반올림	s08
if / else	조건	s09
while True	무한 반복	s10
def	함수 만들기	s12

3차시 · 미니 프로젝트와 수업 적용 (50분)

시간	활동
0~5분	프로젝트 안내, 하나 고르기
5~30분	미니 프로젝트 (택1, 끝나면 다른 것도)
30~40분	센서 이야기: 온도는 왜 높게 나올까?
40~50분	학교급별 수업 적용, 정리, Q&A

프로젝트는 2차시에서 배운 것만 조합합니다. 새 문법은 거의 없습니다.

p01 · 기울이면 화살표 (움직임 센서)

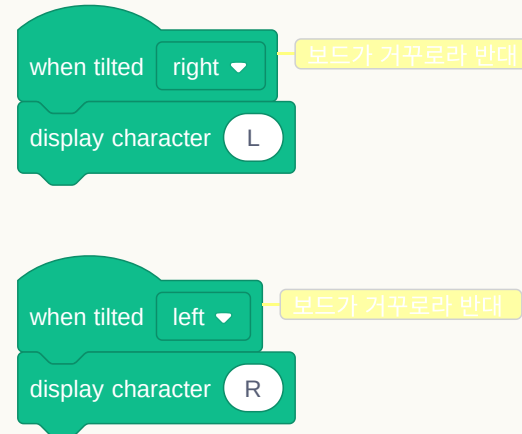
보드를 왼쪽·오른쪽으로 기울이면 그 방향 글자가 나옵니다.

Python

```
from sense_hat import SenseHat
sense = SenseHat()
sense.set_rotation(180)
sense.clear()

while True:
    x = sense.get_accelerometer_raw()["x"]
    if x < -0.3:
        sense.show_letter("R")
    elif x > 0.3:
        sense.show_letter("L")
    else:
        sense.clear()
```

Scratch



- `get_accelerometer_raw()["x"]`: 좌우로 얼마나 기울었는지 (-1 ~ 1). Argon 케이스에서는 **오른쪽으로 기울이면 음수(-)** 입니다.
- `elif`: "그게 아니고 만약 ~라면"
- 쓰인 문법: `while (s10), if (s09), 변수(s02)`
- ▶ 바꿔 보기: 글자 대신 색으로 표시하기
- ⚠ SenseHAT이 거꾸로 꽂혀 있어서 센서의 왼쪽·오른쪽도 반대입니다. Scratch 블록의 `right` 에 `L` 을 넣은 이유입니다.

p02 · 카멜레온 (컬러 센서, SenseHAT V2 전용)

컬러 센서에 물건을 대면 LED 전체가 그 색으로 바뀝니다.

```
from sense_hat import SenseHat
sense = SenseHat()
sense.set_rotation(180)
sense.clear()
sense.colour.gain = 64

while True:
    r, g, b, c = sense.colour.colour
    sense.clear(r, g, b)
```

- `sense.colour.colour`: 빨강·초록·파랑·밝기 4개 값을 한 번에 줌
- `r, g, b, c = ...`: 4개 값을 변수 4개에 나눠 담기
- `gain = 64`: 센서 감도를 최대로 (1, 4, 16, 64 중 하나만 가능)
- 컬러 센서는 LED 아래쪽 작은 창입니다. 물건을 **가까이** 대 보세요.
- 어두우면 색이 잘 안 나옵니다. 휴대폰 손전등으로 물건을 비추면 선명해집니다.
- ▶ 바꿔 보기: 색 이름 맞추기 (`if r > g and r > b:` → 빨강)
- ※ Scratch에는 컬러 센서 블록이 없어서 Python으로만 합니다.

p03 · 전자 주사위 (조이스틱 + 무작위)

조이스틱을 누를 때마다 1~6 중 하나가 나옵니다.

Python

```
from sense_hat import SenseHat
from random import randint
sense = SenseHat()
sense.set_rotation(180)
sense.clear()

while True:
    e = sense.stick.wait_for_event()
    if e.action == "pressed":
        sense.show_letter(str(randint(1, 6)))
```

- `randint(1, 6)`: 1~6 중 아무 수 하나 (새 개념: 무작위)
- 쓰인 문법: 조이스틱(`s11`), `str(s08)`
- ▶ 바꿔 보기: 1~100 뽑기 (`show_message` 사용)

Scratch에서는 **흔들면** 주사위가 나오게 해 봅시다.

Scratch

when shaken

display character pick random 1 to 6

센서 이야기: 온도는 왜 높게 나올까? (10분)

1. 방 온도계와 SenseHAT 온도를 비교해 봅시다. → SenseHAT이 5~15°C 높습니다.
2. 이유: SenseHAT이 라즈베리파이 **CPU 바로 위에** 있어서, CPU 열을 함께 잡니다.
3. 해결 방법 (학생과 토의)
 - 연장 케이블로 SenseHAT을 떨어뜨리기
 - 실제 온도와의 **차이만큼 빼서 보정하기**: `t = sense.get_temperature() - 10`
4. 수업 포인트: "**센서 값을 그대로 믿지 말고, 확인하고 보정한다**" → 과학적 태도

학교급별 수업 적용 아이디어

학교급	추천 활동	도구
초등	LED 픽셀아트 이름표, 흔들면 주사위, 신호등	Scratch
중학교 (정보)	s05~s11로 순차·반복·조건 익히기, 온도 알리미	Scratch → Python
고등학교	교실 온도 측정·기록, 센서 보정 실험, 함수로 코드 정리	Python

수업 팁

- 짝 코딩: 한 명은 입력, 한 명은 읽어 주기 → 5분마다 교대
- 처음에는 **완성 코드를 주고 "바꿔 보기"부터** → 성공 경험이 먼저
- 오류가 나면 **빨간 오류 메시지 마지막 줄**을 같이 읽는 연습
- 수업이 끝나면 `sense.clear()` 로 LED 끄기

정리

- 오늘 한 일: OS 설치 → 한글 설정 → LED·센서·조이스틱 → 미니 프로젝트
- 자료 폴더 `sensehat-workshop/` 를 그대로 수업에 활용하세요.
- 더 공부하기
 - Raspberry Pi 재단 SenseHAT 문서: <https://www.raspberrypi.com/documentation/accessories/sense-hat.html>
 - Astro Pi (학생이 만든 코드를 우주정거장에서 실행): <https://astro-pi.org>

부록 · 더 해 보기: 추가 Python 예제 12개

기초 12단계를 마친 뒤 해 볼 수 있는 예제입니다. 파일 위치: `code/python/extra/` 모든 파일은 첫 네 줄(`import` → `SenseHat()` → `set_rotation(180)` → `clear()`)로 시작하며, 아래에서는 생략합니다. LED 결과 그림은 실제 SenseHAT에서 읽은 장면입니다.

e01 · 무지개 채우기

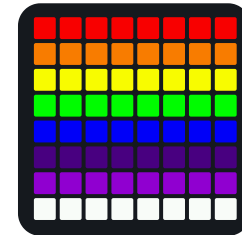
Python · `e01_rainbow.py`

```
from time import sleep          # 기다리기(sleep) 도구 가져오기

# 줄마다 쓸 색 8개: 빨·주·노·초·파·남·보·흰
colors = [(255, 0, 0), (255, 127, 0), (255, 255, 0), (0, 255, 0),
          (0, 0, 255), (75, 0, 130), (148, 0, 211), (255, 255, 255)]

for y in range(8):              # 세로 줄 0~7 (바깥 반복)
    for x in range(8):          # 가로 칸 0~7 (안쪽 반복)
        sense.set_pixel(x, y, colors[y]) # 줄마다 다른 색
        sleep(0.03)
```

LED 결과



새로 나오는 것: 이중 `for`와 리스트

e02 · 웃는 얼굴

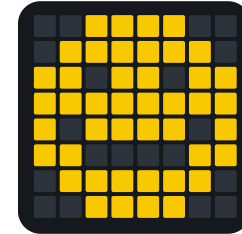
Python · e02_smiley.py

```
Y = (255, 200, 0)    # 노랑
O = (0, 0, 0)        # 꺼짐

# 64칸 목록: 한 줄에 8칸씩, 위에서 아래로 8줄 (모양이 그대로 보임)
face = [
    O, O, Y, Y, Y, Y, O, O,
    O, Y, Y, Y, Y, Y, Y, O,
    Y, Y, O, Y, Y, O, Y, Y,
    Y, Y, Y, Y, Y, Y, Y, Y,
    Y, O, Y, Y, Y, Y, O, Y,
    Y, Y, O, O, O, O, Y, Y,
    O, Y, Y, Y, Y, Y, Y, O,
    O, O, Y, Y, Y, Y, O, O,
]

sense.set_pixels(face)    # 64칸을 한 번에 그리기
```

LED 결과



새로 나오는 것: **set_pixels**로 64칸을 한 번에 그리기

e03 · 두근두근 하트

Python · e03_heart.py

```
from time import sleep          # 기다리기(sleep) 도구 가져오기

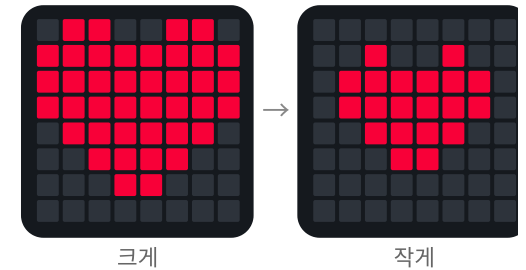
R = (255, 0, 60)               # 빨강
O = (0, 0, 0)                  # 꺼짐

big = [ ... 8줄 × 8칸 그림 (전체는 파일에) ... ]
small = [ ... 8줄 × 8칸 그림 (전체는 파일에) ... ]

for i in range(5):              # 5번 두근두근
    sense.set_pixels(big)        # 크게
    sleep(0.4)
    sense.set_pixels(small)      # 작게
    sleep(0.4)
sense.clear()
```

새로 나오는 것: 그림 두 개를 번갈아 보여 주기

LED 결과



e04 · 카운트다운

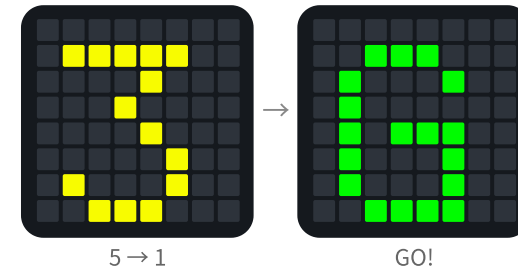
Python · e04_countdown.py

```
from time import sleep          # 기다리기(sleep) 도구 가져오기

for n in range(5, 0, -1):      # range(시작, 끝, 간격): 5, 4, 3, 2, 1 (끝 0은 빠짐)
    # 숫자는 str()로 글자로 바꿔야 보여 줄 수 있음
    sense.show_letter(str(n), text_colour=(255, 255, 0))
    sleep(1)
sense.show_message("GO!", text_colour=(0, 255, 0))
```

새로 나오는 것: **range**를 거꾸로 세기

LED 결과



e05 · 습도 알리미

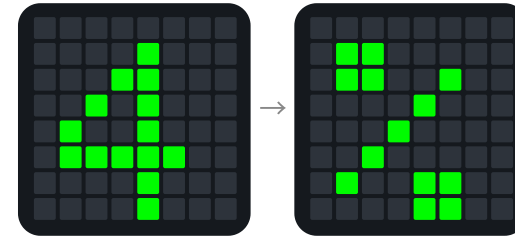
Python · e05_humidity.py

```
from time import sleep          # 기다리기(sleep) 도구 가져오기

while True:
    h = sense.get_humidity()     # 습도(%) 읽기
    if h > 60:
        color = (0, 0, 255)     # 습함: 파랑
    elif h < 30:
        color = (255, 128, 0)   # 건조: 주황
    else:
        color = (0, 255, 0)     # 알맞음: 초록
    # 숫자를 글자로 바꾸고 "%"를 + 로 이어 붙이기
    sense.show_message(str(round(h)) + "%", text_colour=color)
    sleep(1)
```

새로 나오는 것: 습도에 따라 색 바꾸기

LED 결과



예: 43%

e06 · 막대 온도계

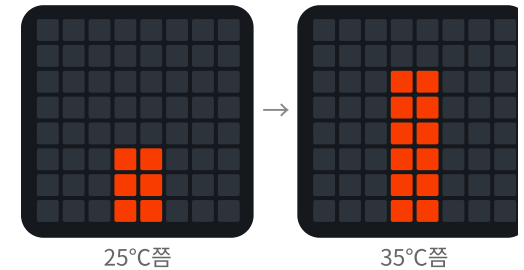
Python · e06_thermometer_bar.py

```
from time import sleep          # 기다리기(sleep) 도구 가져오기

while True:
    t = sense.get_temperature()   # 온도 읽기
    height = int((t - 20) / 20 * 8) # 20°C → 0칸, 40°C → 8칸
    height = max(0, min(8, height)) # 0~8 사이로 자르기
    sense.clear()
    for y in range(8 - height, 8): # 아래부터 채우기
        for x in range(3, 5):      # 가운데 두 칸(x=3, 4) 굵기
            sense.set_pixel(x, y, (255, 60, 0))
    print(round(t, 1), "°C")       # Thonny 아래 셀 창에도 숫자 출력
    sleep(1)
```

새로 나오는 것: 온도를 막대 높이로 (20°C~40°C)

LED 결과



e07 · 조이스틱으로 점 옮기기

Python · e07_joystick_dot.py

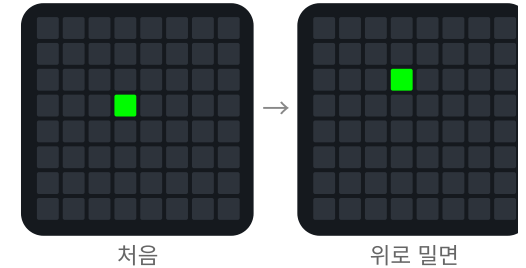
```
# Argon 케이스에서는 SenseHAT이 거꾸로 꽂혀 조이스틱 방향이 반대로 읽힙니다.

x, y = 3, 3                # 점의 처음 위치 (가운데쯤)
sense.set_pixel(x, y, (0, 255, 0))

while True:
    e = sense.stick.wait_for_event()
    if e.action != "pressed":    # 누른 것이 아니면 (땜·계속 누름)
        continue                # 아래는 건너뛰고 다시 기다리기
    if e.direction == "down" and y > 0:    # 위로 밀기 (거꾸로 읽힘)
        y = y - 1
    elif e.direction == "up" and y < 7:    # 아래로 밀기
        y = y + 1
    elif e.direction == "right" and x > 0:  # 왼쪽으로 밀기
        x = x - 1
    elif e.direction == "left" and x < 7:  # 오른쪽으로 밀기
        x = x + 1
    sense.clear()                # 지우고
    sense.set_pixel(x, y, (0, 255, 0))    # 새 위치에 다시 그리기
```

새로 나오는 것: 변수로 위치 기억하기

LED 결과



e08 · 구슬 굴리기

Python · e08_marble.py

```
# Argon 케이스에서는 오른쪽으로 기울이면 x가, 아래로 기울이면 y가 음수(-)입니다.
from time import sleep          # 기다리기(sleep) 도구 가져오기

a = sense.get_accelerometer_raw() # 시작할 때의 기울기를 재서
x0 = a["x"]                     # "평평함"의 기준으로 삼기
y0 = a["y"]                     # (책상이 조금 기울어 있어도 괜찮음)

px = 3                          # 구슬의 가로 위치 (0~7)
py = 3                          # 구슬의 세로 위치 (0~7)
while True:
    a = sense.get_accelerometer_raw()
    x = a["x"] - x0              # 좌우 기울기
    y = a["y"] - y0              # 앞뒤 기울기
    if x < -0.2 and px < 7:      # 오른쪽(동)으로 기울였고, 끝이 아니면
        px = px + 1             # 오른쪽으로
    elif x > 0.2 and px > 0:     # 왼쪽(서)으로 기울였고, 끝이 아니면
        px = px - 1             # 왼쪽으로
    if y < -0.2 and py < 7:      # 아래(남)로 기울였고, 끝이 아니면
        py = py + 1             # 아래로
    elif y > 0.2 and py > 0:     # 위(북)로 기울였고, 끝이 아니면
        py = py - 1             # 위로
    sense.clear()
    sense.set_pixel(px, py, (0, 200, 255)) # 구슬 그리기
    sleep(0.1)                  # 숫자를 줄이면 더 빨리 굴러감
```

LED 결과



새로 나오는 것: 기울이는 쪽(동서남북)으로 구슬이 굴러감

e09 · 흔들면 주사위

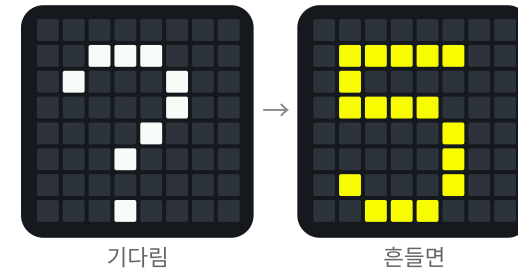
Python · e09_shake_dice.py

```
from random import randint      # 무작위 수 도구 가져오기
from time import sleep         # 기다리기(sleep) 도구 가져오기

sense.show_letter("?")         # 흔들기를 기다리는 중
while True:
    a = sense.get_accelerometer_raw()  # x·y·z 세 방향의 힘 (가만히 두면 1 이하)
    # abs()는 부호(-)를 없앤 크기. 어느 방향이든 1.5보다 크면 흔들 것
    if abs(a["x"]) > 1.5 or abs(a["y"]) > 1.5 or abs(a["z"]) > 1.5:
        sense.show_letter(str(randint(1, 6)), text_colour=(255, 255, 0))
        sleep(1)                # 한 번 흔들 때 한 번만
```

새로 나오는 것: **가속도 크기로 흔들림 알아내기**

LED 결과



e10 · 반응 속도 게임

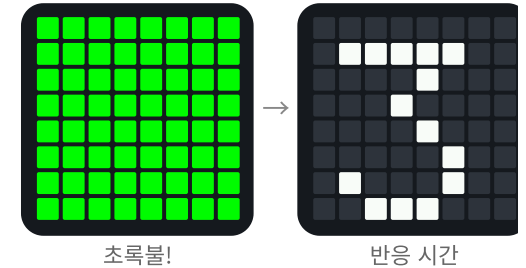
Python · e10_reaction_game.py

```
from random import uniform      # 무작위 소수 도구 가져오기
from time import sleep, time    # 기다리기, 지금 시각(초) 도구

sense.show_message("Ready", text_colour=(255, 255, 0))
sleep(uniform(1, 4))             # 1~4초 중 아무 때나
sense.clear(0, 255, 0)          # 초록불!
start = time()                  # 초록불 켜진 시각 기억
sense.stick.get_events()        # 미리 눌린 기록 지우기
while True:
    e = sense.stick.wait_for_event()
    if e.action == "pressed":
        break                   # 누르면 반복 빠져나가기
ms = round((time() - start) * 1000) # 걸린 시간(초) → 밀리초(ms)
sense.clear()
print(ms, "ms")                 # Thonny 셸 창에도 출력
sense.show_message(str(ms) + "ms")
```

새로 나오는 것: 초록불이 켜지면 조이스틱을 빨리 누르기

LED 결과



e11 · 환경 기록기

Python · e11_data_logger.py

```
from time import sleep, strftime # 기다리기, 시각을 글자로 만들기

f = open("data.csv", "w")          # 파일 열기 ("w" = 새로 쓰기, 이 코드와 같은 폴더)
f.write("time,temp,humidity,pressure\n") # 첫 줄: 제목 (\n = 줄 바꿈)
for i in range(10):                # 10번, 2초마다
    t = round(sense.get_temperature(), 1) # 온도(°C), 소수 첫째 자리까지
    h = round(sense.get_humidity(), 1)    # 습도(%)
    p = round(sense.get_pressure(), 1)    # 기압(hPa)
    # 한 줄 = 시각,온도,습도,기압 (심표로 구분 → 엑셀에서 칸이 나뉨)
    f.write(strftime("%H:%M:%S") + "," + str(t) + "," + str(h) + "," + str(p) + "\n")
    print(i + 1, t, h, p)             # 셀 창에서 확인
    sense.set_pixel(i % 8, 0, (0, 255, 0)) # 기록할 때마다 점 하나
    sleep(2)
f.close()                             # 파일 닫기 (꼭 해야 저장이 마무리됨)
sense.show_message("Saved")
```

새로 나오는 것: 온도·습도·기압을 CSV 파일로 저장 (엑셀로 열기)

LED 결과



e12 · 색 이름 맞추기

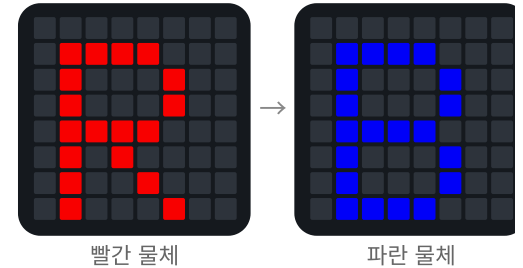
Python · e12_colour_name.py

```
from time import sleep          # 기다리기(sleep) 도구 가져오기

sense.colour.gain = 64          # 컬러 센서 감도 최대로 (1, 4, 16, 64 중)
while True:
    r, g, b, c = sense.colour.colour    # 센서가 본 빨강·초록·파랑·밝기
    if r > g and r > b:                # 빨강이 가장 크면
        sense.show_letter("R", text_colour=(255, 0, 0))
    elif g > r and g > b:              # 초록이 가장 크면
        sense.show_letter("G", text_colour=(0, 255, 0))
    else:                              # 나머지는 파랑으로
        sense.show_letter("B", text_colour=(0, 0, 255))
    print(r, g, b)                    # 셸 창에서 실제 값 확인
    sleep(0.5)
```

새로 나오는 것: 컬러 센서 (SenseHAT V2 전용)

LED 결과



부록 · 게임 4개

조이스틱과 기울기 센서로 즐기는 짧은 게임입니다. 기울기 게임은 시작할 때의 자세를 "평평함"으로 삼으므로, 실행하는 순간에는 기기를 가만히 두세요. 파일 위치: `code/python/game/` 모든 파일은 **첫 네 줄**(`import` → `SenseHat()` → `set_rotation(180)` → `clear()`)로 시작하며, 아래에서는 생략합니다. LED 결과 그림은 실제 SenseHAT에서 읽은 장면입니다.

g01 · 뱀 게임

Python · g01_snake.py

```
# Argon 케이스에서는 조이스틱 방향이 반대로 읽혀서, 움직일 방향을 반대로 적어 둡니다.
from random import randint      # 무작위 정수 도구 가져오기
from time import sleep         # 기다리기(sleep) 도구 가져오기

# 조이스틱이 알려 준 방향 → 실제로 움직일 칸 (x 변화, y 변화)
move = {"up": (0, 1), "down": (0, -1), "left": (1, 0), "right": (-1, 0)}

snake = [(3, 4), (2, 4)]      # 뱀 몸통 좌표 목록 (맨 앞이 머리)
dx, dy = 1, 0                # 처음에는 오른쪽으로 감
apple = (6, 2)               # 사과 위치
turns = []                   # 눌렀지만 아직 쓰지 않은 방향 (빨리 두 번 눌러도 빠지지 않게)

while True:
    for e in sense.stick.get_events():      # 그동안 누른 조이스틱 기록
        if e.action == "pressed" and e.direction in move:
            turns.append(move[e.direction])

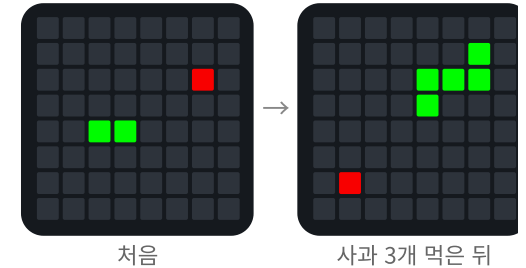
    while turns:
        ndx, ndy = turns.pop(0)
        if (ndx, ndy) != (dx, dy) and (ndx, ndy) != (-dx, -dy):  # 같은 방향·정반대는 건너뛴
            dx, dy = ndx, ndy
            break

    x, y = snake[0]          # 지금 머리 위치
    head = ((x + dx) % 8, (y + dy) % 8)  # 새 머리 (벽을 넘으면 반대편으로)
    if head in snake:        # 내 몸에 부딪히면
        break               # 게임 끝
    snake.insert(0, head)    # 앞에 머리 붙이기
    if head == apple:        # 사과를 먹었으면 꼬리를 그대로 뒤서 길어짐
        while apple in snake:
            apple = (randint(0, 7), randint(0, 7))
    else:
        snake.pop()         # 안 먹었으면 꼬리 한 칸 떼기

    sense.clear()
    for x, y in snake:
        sense.set_pixel(x, y, (0, 255, 0))  # 뱀은 초록
    sense.set_pixel(apple[0], apple[1], (255, 0, 0))  # 사과는 빨강
    sleep(0.3)

score = len(snake) - 2      # 먹은 사과 수
print("점수:", score)      # Thonny 셸 창에도 출력
sense.show_message("Score " + str(score), text_colour=(255, 255, 0))
```

LED 결과



게임 방법: 조이스틱으로 뱀을 움직여 빨간 사과 먹기

g02 · 돌 피하기

Python · g02_dodge.py

```
# Argon 케이스에서는 오른쪽으로 기울이면 x가 음수 (-)입니다.
from random import randint      # 무작위 정수 도구 가져오기
from time import sleep          # 기다리기(sleep) 도구 가져오기

x0 = sense.get_accelerometer_raw()["x"] # 시작할 때 기울기를 "평평함"으로
px = 3                             # 내 위치 (맨 아래줄, 0~7)
rocks = []                          # 돌 목록 [x, y]
score = 0                           # 피한 돌 수

while True:
    x = sense.get_accelerometer_raw()["x"] - x0 # 좌우 기울기
    if x < -0.2 and px < 7:                     # 오른쪽으로 기울었으면
        px = px + 1
    elif x > 0.2 and px > 0:                     # 왼쪽으로 기울었으면
        px = px - 1

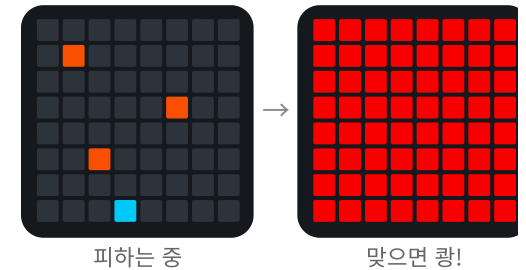
    new = []                                     # 한 칸씩 떨어진 돌들을 담은 새 목록
    for r in rocks:
        if r[1] < 7:                             # 아직 바닥이 아니면 한 칸 아래로
            new.append([r[0], r[1] + 1])
        else:                                     # 바닥을 지나갔으면 피한 것!
            score = score + 1
    rocks = new
    if randint(1, 3) == 1:                         # 세 번에 한 번꼴로 새 돌
        rocks.append([randint(0, 7), 0])

    if [px, 7] in rocks:                          # 돌에 맞으면
        break                                     # 게임 끝

    sense.clear()
    for r in rocks:
        sense.set_pixel(r[0], r[1], (255, 80, 0)) # 돌은 주황
        sense.set_pixel(px, 7, (0, 200, 255))      # 나는 하늘색
        sleep(max(0.08, 0.25 - score * 0.005))    # 점수가 오를수록 빨라짐

sense.clear(255, 0, 0)                          # 광! 빨간 화면
sleep(0.5)
print("점수:", score)                             # Thonny 셸 창에도 출력
sense.show_message("Score " + str(score), text_colour=(255, 255, 0))
```

LED 결과



게임 방법: 좌우로 기울여 떨어지는 돌을 피하기

g03 · 미로 탈출

Python · g03_maze.py

```
# Argon 케이스에서는 오른쪽으로 기울이면 x가, 아래로 기울이면 y가 음수(-)입니다.
from time import sleep, time      # 기다리기, 지금 시각(초) 도구

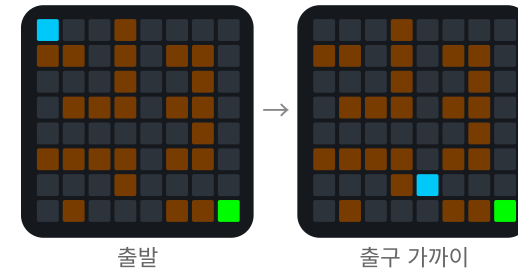
# 미로 지도: # 은 벽, . 은 길, G 는 출구 (한 줄이 LED 한 줄)
maze = [
    "...#....",
    "##.##.",
    "...#..",
    ".###.##.",
    ".....#",
    "####.##.",
    "...#....",
    ".#...##G",
]

a = sense.get_accelerometer_raw()  # 시작할 때 기울기를 "평평함"으로
x0 = a["x"]
y0 = a["y"]
px, py = 0, 0                      # 구슬은 왼쪽 위에서 출발
start = time()                     # 출발 시각 기억

while maze[py][px] != "G":         # 출구에 닿을 때까지 반복
    a = sense.get_accelerometer_raw()
    x = a["x"] - x0                # 좌우 기울기
    y = a["y"] - y0                # 앞뒤 기울기
    nx, ny = px, py                # 가려는 칸 (처음엔 제자리)
    if x < -0.2 and px < 7:         # 오른쪽(동)
        nx = px + 1
    elif x > 0.2 and px > 0:        # 왼쪽(서)
        nx = px - 1
    if y < -0.2 and py < 7:        # 아래(남)
        ny = py + 1
    elif y > 0.2 and py > 0:        # 위(북)
        ny = py - 1
    if maze[ny][nx] != "#":         # 벽이 아니면 대각선으로 한 번에
        px, py = nx, ny
    elif maze[py][nx] != "#":      # 대각선이 막혔으면 가로로만
        px = nx
    elif maze[ny][px] != "#":      # 아니면 세로로만
        py = ny

for y in range(8):                 # 미로 그리기
    for x in range(8):
        if maze[y][x] == "#":
            sense.set_pixel(x, y, (120, 60, 0))  # 벽은 갈색
        elif maze[y][x] == "G":
            sense.set_pixel(x, y, (0, 255, 0))   # 출구는 초록
        else:
```

LED 결과



```
        sense.set_pixel(x, y, (0, 0, 0))      # 길은 검정
sense.set_pixel(px, py, (0, 200, 255))      # 구슬은 하늘색
sleep(0.15)                                # 숫자를 줄이면 더 빨리 굴러감

sec = round(time() - start, 1)    # 걸린 시간(초), 소수 첫째 자리까지
print("탈출!", sec, "초")        # Thonny 셸 창에도 출력
sense.show_message(str(sec) + "s", text_colour=(0, 255, 0))
```

게임 방법: 동서남북으로 기울여 구슬을 초록 출구까지

g04 · 순서 기억 게임

Python · g04_simon.py

```
# Argon 케이스에서는 조이스틱 방향이 반대로 읽혀서, 읽은 방향을 뒤집어 씁니다.
from random import choice      # 목록에서 아무거나 고르는 도구
from time import sleep        # 기다리기(sleep) 도구 가져오기

flip = {"up": "down", "down": "up", "left": "right", "right": "left"}  # 거꾸로 읽힘 바로잡기
colour = {"up": (255, 0, 0), "down": (0, 255, 0), "left": (0, 0, 255), "right": (255, 255, 0)}
spot = {                        # 방향마다 켜 네 칸
    "up":    [(3, 0), (4, 0), (3, 1), (4, 1)],
    "down":  [(3, 6), (4, 6), (3, 7), (4, 7)],
    "left":  [(0, 3), (1, 3), (0, 4), (1, 4)],
    "right": [(6, 3), (7, 3), (6, 4), (7, 4)],
}

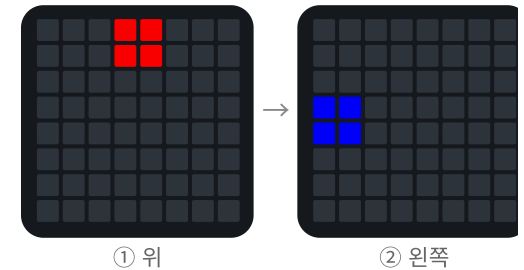
def show(d, t):                # 방향 d 자리를 t초 동안 켜기
    for x, y in spot[d]:
        sense.set_pixel(x, y, colour[d])
    sleep(t)
    sense.clear()

order = []                    # 기억해야 할 순서
ok = True                    # 지금까지 다 맞혔는지
while ok:
    order.append(choice(["up", "down", "left", "right"]))  # 하나 늘리기
    sleep(0.8)
    for d in order:          # 순서 보여 주기
        sleep(0.2)
        show(d, 0.5)

    sense.stick.get_events()  # 보는 동안 눌린 기록 지우기 (마지막 불이 꺼지자마자 누른 것부터 받음)
    for d in order:          # 같은 순서로 밀었는지 하나씩 확인
        e = sense.stick.wait_for_event()
        while e.action != "pressed" or e.direction not in flip:
            e = sense.stick.wait_for_event()  # 누른 것이 아니면 다시 기다리기
        pushed = flip[e.direction]           # 실제로 밀 방향
        show(pushed, 0.2)
        if pushed != d:                      # 틀리면
            ok = False
            break

    score = len(order) - 1                  # 다 맞힌 판 수
    sense.clear(255, 0, 0)                 # 틀렸다! 빨간 화면
    sleep(0.5)
    print("점수:", score)                  # Thonny 셸 창에도 출력
    sense.show_message("Score " + str(score), text_colour=(255, 255, 0))
```

LED 결과



게임 방법: 켜진 순서대로 조이스틱을 밀기, 한 판마다 하나씩 늘어남